

```

def is_Primitive_root(alpha, q):
    result = set()

    for power in range(1, q):
        value = pow(alpha, power, q)
        result.add (value)

    if len(result) == q -1:
        return True
    else:
        return False

def find_Primitive_roots(q):
    Primitive_roots = []

    for alpha in range(2 ,q):
        if is_Primitive_root(alpha,q):
            Primitive_roots.append(alpha)

    return Primitive_roots

def compute_public_key(alpha, private_key, q):
    return pow(alpha, private_key, q)

def compute_shared_key(received_public_key, private_key, q):
    return pow(received_public_key, private_key, q)

q = 71
print("Prime number (q):", q)

Primitive_roots = find_Primitive_roots(q)
print("\nPrimitive roots of",q,"are:",Primitive_roots)

alpha = Primitive_roots[0]
print("\nselected Primitive root (alpha):", alpha)

Xa = 19
Xb = 23
print("\nprivate keys:")
print("Xa (Private key of User A):", Xa)
print("Xb (Private key of User B):", Xb)

Ya = compute_public_key(alpha, Xa, q)
Yb = compute_public_key(alpha, Xb, q)
print("\nPublic Keys:")
print("Ya (Public key of User A):", Ya)
print("Yb (Public key of User B):", Yb)

Ka = compute_shared_key(Yb, Xa, q)
Kb = compute_shared_key(Ya, Xb, q)

```

```
print("\nShared Secret Keys:")
print("Ka (Computed by User A):", Ka)
print("Kb (Computed by User B):", Kb)

if Ka == Kb:
    print("\nkey Exchange Succeseful!")
else:
    print("\nkey Exchange Failed!")
```